

MARS ROVER MANIPAL RESEARCH AI TASKPHASE Report 8

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Principal Component Analysis (PCA)

1.1 Data preparation and cleaning

```
1 df = pd.read_csv('titanic.csv')
2 df.head()
✓ 0.0s
```

- The `df` DataFrame stores the dataset from `titanic.csv`.
- `df.head()` prints the first five rows.

```
1 numeric_cols = ['Age', 'Fare', 'SibSp', 'Parch', 'Pclass']
2 data = df[numeric_cols].copy().fillna(df[numeric_cols].mean())
3
4 X = data.values.astype(float)
✓ 0.0s
```

- The `numeric_cols` list selects the numerical columns. For this dataset, the numerical columns are Age, Fare, SibSp, Parch, Pclass.
- The `data` DataFrame is the cleaned version of `df`.
 - `df[numeric_cols]`: It selects only the columns listed in Line 1 from the original DataFrame `df`.
 - `.copy()`: It creates an independent deep copy of this subset. This ensures that changes made to the new variable `data` do not accidentally alter the original `df`.
 - `.fillna(df[numeric_cols].mean())`: This handles missing data. It finds every NaN value in the selected columns and replaces it with the mean (average) of that specific column.

- The `X` variable stores the cleaned DataFrame `data` and stores it in a NumPy array of data type `float`. It represents the **feature matrix**.

1.2 Z-score scaling

```
1  # manual Z-score
2
3  X_mean = X.mean(axis=0)
4  X_std = X.std(axis=0)
5  X_std = np.where(X_std == 0, 1, X_std)
6
7  Xs = (X - X_mean) / X_std
```

✓ 0.0s

- `X_mean` and `X_std` store the mean and standard deviation for `X` downwards with squashed rows (`axis=0`).
- A safety mechanism is used for `X_std`.
 - If any standard deviation is exactly 0, it replaces that 0 with 1.
 - Otherwise, it keeps the original value.
 - This helps in preventing a division by zero error.
- `Xs` is the scaled version of the feature matrix using Z-score.

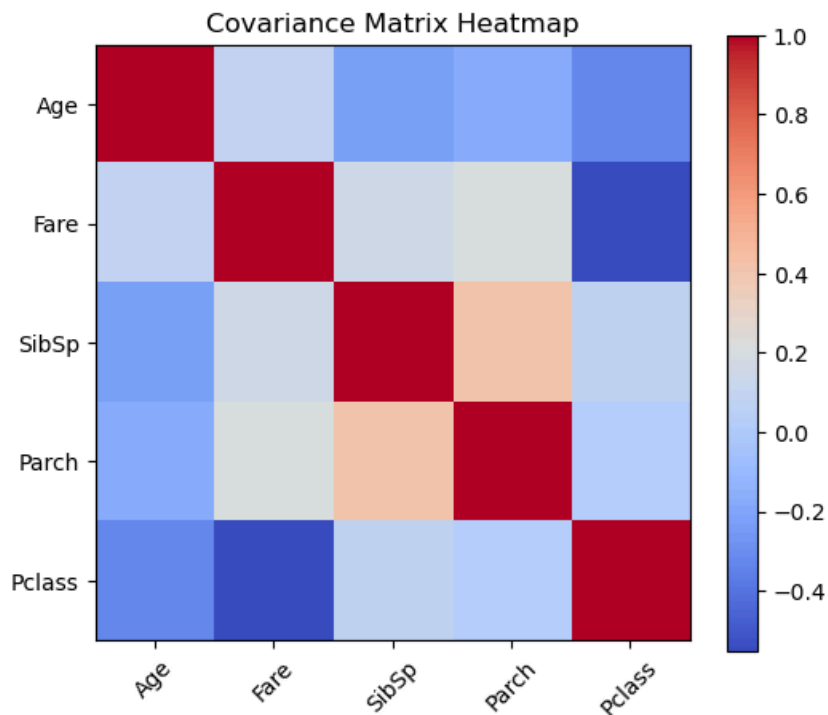
1.3 Finding the covariance matrix

```
1 # covariance matrix
2
3 cov = np.cov(Xs.T)
4 cov
✓ 0.0s
```

- The `cov` variable stores the covariance matrix.
 - `Xs` is the scaled matrix from Z-score.
 - `.T` transposes the matrix.
 - The `np.cov()` function calculates the covariance between the features.
 - This data is mathematically equal to the correlation matrix as the data is already standardized using Z-score.
 - The values range between -1 and 1.

```
1 # plotting the covariance matrix heatmap
2
3 plt.figure(figsize=(6,5))
4 plt.imshow(cov, cmap="coolwarm")
5 plt.colorbar()
6 plt.xticks(range(len(numeric_cols)), numeric_cols, rotation=45)
7 plt.yticks(range(len(numeric_cols)), numeric_cols)
8 plt.title("Covariance Matrix Heatmap")
9 plt.show()
✓ 0.2s
```

- This is the code used to visualize the covariance matrix as a heatmap for better readability. The `matplotlib` library is used (`plt`).
- The `cov` variable signifies the covariance matrix.



1.4 Calculating eigenvalues and eigenvectors

```

1 eigenvalues, eigenvectors = np.linalg.eig(cov)
2 idx = eigenvalues.argsort()[::-1] # sort in descending order
3
4 eigenvalues_sorted = eigenvalues[idx]
5 eigenvectors_sorted = eigenvectors[:, idx]
6
7 explained_variance_ratio = eigenvalues_sorted / eigenvalues_sorted.sum()
8
9 eigenvalues_sorted, explained_variance_ratio
✓ 0.0s

```

- The eigenvalues and eigenvectors of the covariance matrix are calculated using the `np.linalg.eig()` function.
- The `idx` variable is a list of indices to sort the data from highest to lowest.
 - The `.argsort()` function sorts the array in ascending order.
 - `[::-1]` reverses the order.
 - This is important as for PCA we need to consider the component with the largest variance as the most important component.
- `eigenvalues_sorted` is a list of the sorted eigenvalues.
- `eigenvectors_sorted` is a matrix where the columns of the eigenvector matrix match the sorted eigenvalues.

- The `explained_variance_ratio` calculates the percentage of information (variance) each principal component holds.

$$Ratio_i = \frac{\lambda_i}{\sum_{i=1}^n \lambda_i}$$

1.5 Dimensionality reduction

```

1 # projection on k components
2
3 k = 2
4 PCs = Xs @ eigenvectors_sorted[:, :k]
5
6 PCs[:10]
✓ 0.0s

```

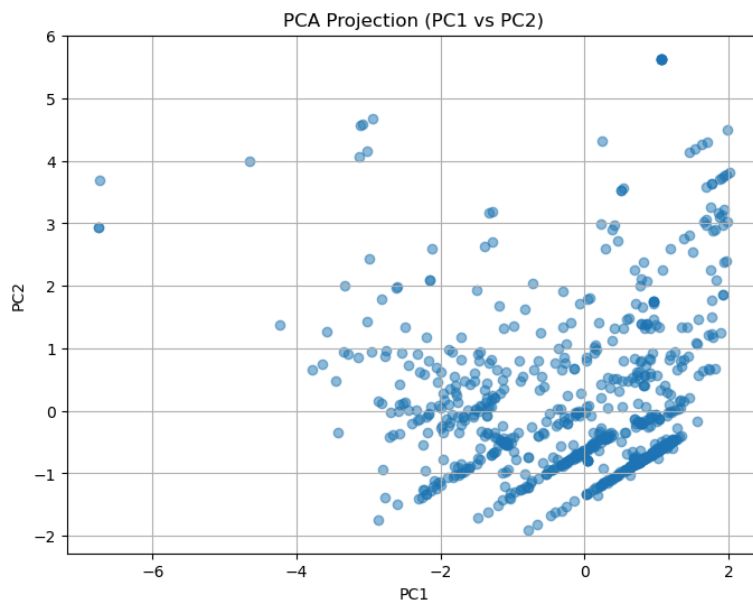
- `k = 2` indicates that there are two new dimensions or Principal Components.
- The `PCs` matrix stores the result of the matrix multiplication of `Xs` and the weight matrix.
 - The weight matrix is created by taking all rows of the sorted eigenvectors and taking only the first `k` (first two) columns.
 - This creates a weight matrix of shape `5 x 2`.
- `PCs[:10]` displays the first ten rows of the new matrix.

1.6 PC1 vs PC2 plot

```

1 # plot of PC1 vs PC2
2
3 plt.figure(figsize=(8,6))
4 plt.scatter(PCs[:,0], PCs[:,1], alpha=0.5)
5 plt.xlabel("PC1")
6 plt.ylabel("PC2")
7 plt.title("PCA Projection (PC1 vs PC2)")
8 plt.grid(True)
9 plt.show()
✓ 0.0s

```



- The distinct lines likely represent groups of passengers with the exact same integer value for one major feature.
- There are no two clearly separated blobs. The points overlap significantly in the center.
- The isolated dots at the top right or far left represent outliers.

2. SVM and Logistic Regression

2.1 Preparing target variable

```
1 y = df['Survived'].copy().values
2 y = np.where(y == 1, 1, -1) # map to {-1, +1}
✓ 0.0s
```

- The variable `y` is a Series of only the Survived column. The `.values` converts it into a NumPy array.
- It maps `{0, 1}` to `{-1, 1}`.

2.2 Train test split

```
1 # train test split (80/20)
2
3 def train_test_split(X, y, test_size=0.2, shuffle=True, seed=67):
4     n = X.shape[0]
5     idx = np.arange(n)
6     if shuffle:
7         rng = np.random.RandomState(seed)
8         rng.shuffle(idx)
9     split = int(n * (1 - test_size))
10    train_idx, test_idx = idx[:split], idx[split:]
11    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
12
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
14 print("train/test sizes:", X_train.shape[0], X_test.shape[0])
✓ 0.0s
```

- The `train_test_split` function has five parameters:
 - `X`: feature matrix
 - `y`: target vector
 - `test_size`: 0.2 for 80/20 split
 - `seed`: ensures reproducibility
- `n` is the total number of passengers (rows).
- `idx` creates a list of indices from 0 to `n-1`. These IDs are manipulated instead of moving the massive data arrays around directly.
- The list of indices is randomly shuffled.
- The `split` variable is an integer which is the exact index of splitting.
- `train_idx` and `test_idx` are the first 80% and remaining 20% of the shuffled IDs respectively.
- The function returns four separate arrays:
 - Features for training
 - Features for testing
 - Labels for training
 - Labels for testing

2.3 Standardizing features

```
1 # standardize features
2
3 mean = X_train.mean(axis=0)
4 std = X_train.std(axis=0)
5 std = np.where(std == 0, 1.0, std)
6
7 X_train_std = (X_train - mean) / std
8 X_test_std = (X_test - mean) / std
```

Using Z-score, the training and testing data can be standardized.

2.4 Defining the Linear SVM class

```
1 # linear SVM (hinge loss) implemented with mini-batch SGD
2
3 class LinearSVM:
4     def __init__(self, lr=0.01, lambda_reg=1e-4, n_epochs=100, batch_size=32):
5         self.lr = lr
6         self.lambda_reg = lambda_reg
7         self.n_epochs = n_epochs
8         self.batch_size = batch_size
9
10        self.w = None
11        self.b = 0.0
```

- The constructor defines the properties of the SVM.
 - `lr`: learning rate
 - `lambda_reg`: regularization
 - `n_epochs`: number of epochs
 - `batch_size`: batch size
 - `w`: weights
 - `b`: biases


```

14 def _hinge_gradient(self, X_batch, y_batch):
15     n = X_batch.shape[0]
16     margins = y_batch * (X_batch.dot(self.w) + self.b)
17
18     mask = margins < 1.0
19
20     if mask.sum() == 0:
21         grad_w = self.lambda_reg * self.w
22         grad_b = 0.0
23         return grad_w, grad_b
24
25     X_m = X_batch[mask]
26     y_m = y_batch[mask][:, np.newaxis]
27
28     grad_hinge_w = - (X_m * y_m).mean(axis=0)
29     grad_hinge_b = - (y_m.flatten()).mean()
30
31     grad_w = self.lambda_reg * self.w + grad_hinge_w
32     grad_b = grad_hinge_b
33
34     return grad_w, grad_b

```

- This is a helper method. It takes a mini-batch of data (eg. 32 passengers) instead of the entire dataset for faster calculation.
- n denotes the total number of passengers.
- The functional margin for each passenger is denoted by the margins variable.
$$\text{Margin} = y \cdot (w \cdot X + b)$$
- The mask variable is a boolean filter. This is because we only care about the points that are:
 - misclassified (margin > 0)
 - correct but too close to boundary ($0 < \text{margin} < 1$)
 - If the point is correctly classified and far (margin > 1), the gradient is zero. This is ignored by the model.
- An optimization check is run.
 - If every passenger is perfectly classified (margin > 1), the classification error does not need to be fixed.
 - w is shrunk slightly to prevent overfitting.
- X_m and y_m contain only the passengers the model got wrong.
- grad_hinge_w indicates the hinge loss. Calculates the direction to move the weights to fix the errors.
- grad_w combines regularization ($\text{lambda_reg} * w$) and hinge loss (grad_hinge_w).

```

def fit(self, X, y):
    n_samples, n_features = X.shape

    self.w = np.zeros(n_features, dtype=float)
    self.b = 0.0

    for epoch in range(1, self.n_epochs + 1):
        idx = np.arange(n_samples)
        np.random.shuffle(idx)
        X_shuffled = X[idx]
        y_shuffled = y[idx]

        # mini-batch SGD
        for start in range(0, n_samples, self.batch_size):
            end = start + self.batch_size
            X_batch = X_shuffled[start:end]
            y_batch = y_shuffled[start:end]

            grad_w, grad_b = self._hinge_gradient(X_batch, y_batch)

            # gradient
            self.w -= self.lr * grad_w
            self.b -= self.lr * grad_b

```

- `X.shape`: Detects the `n_samples` (891 passengers) and `n_features` (2 PCA components).
- `np.zeros`: Initializes the weights (`w`) to `[0, 0]`. The model learns from scratch.
- **Epoch loop**
 - Shuffling is used to mix up the order of passengers before every epoch.
 - This is crucial for Stochastic Gradient Descent (SGD).
 - **Mini-Batch loop**
 - Mini-Batch is used as computing gradients on 32 items is faster than the full sample size (891).
 - Gradients are calculated using the `_hinge_gradient` function.
 - The weights and biases are updated by subtracting them from the product of the learning rate and gradient.

```
def decision_function(self, X):
    return X.dot(self.w) + self.b

def predict(self, X):
    scores = self.decision_function(X)
    return np.where(scores ≥ 0, 1, -1)
```

- **Decision function**

- It calculates the raw score for each passenger.

$$f(x) = w \cdot X + b$$

- **Predict function**

- It converts the final scores into labels (-1 or 1).
- If the score is positive, the model predicts 1 (survived).
- If the score is negative, the model predicts -1 (not survived).

2.5 Defining the Logistic Regression class

```
1 # logistic regression
2
3 class LogisticRegression:
4     def __init__(self, lr=0.1, n_epochs=200, batch_size=None, lambda_reg=0.0):
5         self.lr = lr
6         self.n_epochs = n_epochs
7         self.batch_size = batch_size
8         self.lambda_reg = lambda_reg
9
10        self.w = None
11        self.b = 0.0
12
13        self.history = {'loss': []}
```

- The constructor defines the properties of the logistic regression model.
 - `lr`: learning rate
 - `lambda_reg`: regularization (turned off by default)
 - `n_epochs`: number of epochs
 - `batch_size`: batch size (none, uses batch instead of mini batch)
 - `w`: weights
 - `b`: biases
 - The `history` dictionary stores the values of `loss` at every epoch. It is initially an empty list.

```
@staticmethod
def _sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))
```

- The method is a static method. This means that it does not need access to `self` (an instance of the class).
- The `_sigmoid` function is a helper method which uses a mathematical formula to calculate the probability.
- The value of `z` is clipped. This means that all input values are forced to stay between -500 and 500. This is used to prevent overflow.
- The function returns the sigmoid output.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

```
def _compute_loss(self, X, y):
    n = X.shape[0]
    logits = X.dot(self.w) + self.b
    probs = self._sigmoid(logits)

    eps = 1e-12
    probs = np.clip(probs, eps, 1 - eps)
    ce = -(y * np.log(probs) + (1 - y) * np.log(1 - probs)).mean()
    reg = 0.5 * self.lambda_reg * np.sum(self.w ** 2)

    return ce + reg
```

- This function calculates the loss function (log loss).
- The `logits` indicate the raw linear score ($z = w \cdot X + b$).
- `probs` indicate the probabilities. This is calculated from the sigmoid function.
- It is clipped later to avoid $\log(0) = -\text{infinity}$.
- `ce` stores the cost function.

- $J(\theta) = -\frac{1}{m} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$

- L2 regularization is used.
- Returns the total cost (error + complexity penalty).

```
def _gradients(self, X_batch, y_batch):
    n = X_batch.shape[0]
    logits = X_batch.dot(self.w) + self.b
    probs = self._sigmoid(logits)

    error = probs - y_batch
    dw = (X_batch.T @ error) / n
    db = error.mean()

    return dw, db
```

- This helper method calculates the gradients.
- The `logits` indicate the raw linear score ($z = w \cdot X + b$).
- `probs` indicate the probabilities. This is calculated from the sigmoid function.
- The error is indicated by `error = prediction - actual`.
- Returns `dw` and `db`, which are the weight and bias gradients.

```

def fit(self, X, y):
    n_samples, n_features = X.shape

    self.w = np.zeros(n_features, dtype=float)
    self.b = 0.0

    if self.batch_size is None: batch_size = n_samples
    else: batch_size = int(self.batch_size)

    for epoch in range(1, self.n_epochs + 1):
        perm = np.random.permutation(n_samples)
        X_shuf = X[perm]
        y_shuf = y[perm]

        # mini-batch
        for start in range(0, n_samples, batch_size):
            end = start + batch_size
            X_batch = X_shuf[start:end]
            y_batch = y_shuf[start:end]
            dw, db = self._gradients(X_batch, y_batch)

            self.w -= self.lr * dw
            self.b -= self.lr * db

        loss = self._compute_loss(X, y)
        self.history['loss'].append(loss)

```

- If the `batch_size` is `None`, the function uses Batch Gradient Descent. Else, it uses Mini-Batch.
- **Epoch loop**
 - `np.random.permutation` gives a shuffled list of IDs.
 - `X_shuf` and `y_shuf` rearrange the data randomly.
 - **Mini-Batch loop**
 - Slices out a small chunk of data (`X_batch`, `y_batch`).
 - Uses the `_gradients` method to calculate the gradients.

- The weights and biases are updated by subtracting them from the product of the learning rate and gradient.
- At the end of every epoch, total error (log loss) is computed and appended to the `history` dictionary.

```
def predict_probability(self, X):
    logits = X.dot(self.w) + self.b
    return self._sigmoid(logits)

def predict(self, X, threshold=0.5):
    return (self.predict_probability(X) ≥ threshold).astype(int)
```

- **Predict probability function**

- It calculates the raw score for each passenger (logits).

$$f(x) = w \cdot X + b$$

- It then passes that raw score to the `sigmoid` function and returns the output.

- **Predict function**

- If the probability is greater than 0.5, the passenger has survived.
- Else, the passenger has not survived.

2.6 Calculating metrics

```
1 # evaluate metrics
2
3 def metrics(y_true, y_pred):
4     tp = ((y_true==1)&(y_pred==1)).sum()
5     tn = ((y_true==0)&(y_pred==0)).sum()
6     fp = ((y_true==0)&(y_pred==1)).sum()
7     fn = ((y_true==1)&(y_pred==0)).sum()
8     acc = (tp+tn) / len(y_true)
9     prec = tp/(tp+fp) if(tp+fp)>0 else 0
10    rec = tp/(tp+fn) if(tp+fn)>0 else 0
11    f1 = 2*prec*rec/(prec+rec) if(prec+rec)>0 else 0
12    return {"accuracy":acc,"precision":prec,"recall":rec,"f1":f1}
```

- The function calculates the following metrics:
 - tp: true positives
 - tn: true negatives
 - fp: false positives
 - fn: false negatives
 - acc: accuracy
 - prec: precision
 - rec: recall
 - f1: F1 score

2.7 Training both the models

2.7.1 Without PCA

```
1 # training both models without PCA
2
3 logreg = LogisticRegression(lr=0.1, n_epochs=300, batch_size=32)
4 logreg.fit(X_train_std, y_train)
5 lr_pred_test = logreg.predict(X_test_std)
6
7 svm = LinearSVM(lr=0.01, lambda_reg=1e-3, n_epochs=200)
8 svm.fit(X_train_std, y_train)
9 svm_pred_test = svm.predict(X_test_std)
10
11 print("Logistic Regression (No PCA)", metrics(y_test, lr_pred_test))
12 print("SVM (No PCA)", metrics(y_test, svm_pred_test))
```


- `logreg` is the logistic regression object.
 - It has the following parameters:
 - Learning rate (`lr`): 0.1
 - Number of epochs (`n_epochs`): 300
 - Batch size (`batch_size`): 32
 - The logistic regression model is trained on standardized training data.
 - It is then used to predict the test labels.
- `svm` is the linear SVM object.
 - It has the following parameters:
 - Learning rate (`lr`): 0.01
 - Regularization (`lambda_reg`): 1e-3 (0.001)
 - Number of epochs (`n_epochs`): 200
 - The SVM model is trained on standardized training data.
 - It is then used to predict test labels.
- The metrics are then printed.

2.7.2 With PCA

First, we fit PCA on training data.

```

1  # PCA fit on training data
2
3  k = 2
4
5  cov_train = np.cov(X_train_std.T)
6  eigvals_train, eigvecs_train = np.linalg.eig(cov_train)
7  idx_train = eigvals_train.argsort()[::-1]
8  eigvals_sorted_train = eigvals_train[idx_train]
9  eigvecs_sorted_train = eigvecs_train[:, idx_train]
10
11 explained_variance_ratio_train = eigvals_sorted_train / eigvals_sorted_train.sum()
12 print("Explained variance ratios (train):", explained_variance_ratio_train[:k])
13 print(f"Total explained by first {k} components: {explained_variance_ratio_train[:k].sum():.4f}")
14
15 X_train_pca = X_train_std @ eigvecs_sorted_train[:, :k]
16 X_test_pca = X_test_std @ eigvecs_sorted_train[:, :k]
17
18 print("X_train_pca shape:", X_train_pca.shape)
19 print("X_test_pca shape :", X_test_pca.shape)

```

- `k = 2` indicates we need two principal components.
- `cov_train` indicates the covariance matrix of the standardized training data.
- `eigvals_train` and `eigvecs_train` are the eigenvalues and eigenvectors respectively.
- The eigenvalues are then sorted in descending order.

- The eigenvectors are sorted accordingly.
- The `explained_variance_ratio_train` denotes the explained variance ratio of the training data.
- The training and test data is projected onto the first k principal components (2 in this case).

Training the logistic regression model using PCA,

```

1 # train logistic regression on PCA features
2
3 logreg_pca = LogisticRegression(lr=0.1, n_epochs=300, batch_size=32, lambda_reg=0.0)
4 logreg_pca.fit(X_train_pca, (y_train == 1).astype(int))
5
6 lr_pca_pred_test = logreg_pca.predict(X_test_pca)
7
8 print("Logistic Regression (with PCA) metrics:", metrics((y_test==1).astype(int), lr_pca_pred_test))
✓ 0.1s

```

- The `logreg_pca` object is the object for logistic regression with PCA.
 - It has the following parameters:
 - Learning rate (`lr`): 0.1
 - Number of epochs (`n_epochs`): 300
 - Batch size (`batch_size`): 32
 - No regularization (`lambda_reg = 0.0`)
 - The model is trained on the PCA-projected training set.
 - The model predicts the labels.
- The metrics are printed.

Training the SVM model using PCA,

```

1 # train SVM on PCA features
2
3 y_train_svm = np.where(y_train == 1, 1, -1)
4 y_test_svm = np.where(y_test == 1, 1, -1)
5
6 svm_pca = LinearSVM(lr=0.01, lambda_reg=1e-3, n_epochs=200, batch_size=32)
7 svm_pca.fit(X_train_pca, y_train_svm)
8
9 svm_pca_pred_test = svm_pca.predict(X_test_pca)
10 svm_pca_pred_test_bin = (svm_pca_pred_test == 1).astype(int)
11
12 print("SVM (with PCA) metrics:", metrics((y_test==1).astype(int), svm_pca_pred_test_bin))

```

- First, the labels are converted from $\{0, 1\}$ to $\{-1, 1\}$.
- The `svm_pca` object is the object for SVM with PCA.

- It has the following parameters:
 - Learning rate (lr): 0.01
 - Regularization (lambda_reg): 1e-3 (0.001)
 - Number of epochs (n_epochs): 200
 - Batch size (batch_size): 32
- The model is trained on PCA-projected training features with labels in {-1, 1}.
- Predicts using PCA-reduced test features. Predictions will be in {-1, 1}.
- The predictions are converted to binary 0/1 for the metrics.
- The metrics are printed.

2.8 Print summary table

```

1 # summary table
2
3 rows = [
4     [
5         "Logistic (NO PCA)",
6         *metrics((y_test=1).astype(int), lr_pred_test).values()
7     ],
8     [
9         "SVM (NO PCA)",
10        *metrics((y_test=1).astype(int), svm_pred_test).values()
11    ],
12    [
13        "Logistic (WITH PCA)",
14        *metrics((y_test=1).astype(int), lr_pca_pred_test).values()
15    ],
16    [
17        "SVM (WITH PCA)",
18        *metrics((y_test=1).astype(int), svm_pca_pred_test_bin).values()
19    ],
20 ]
21
22 summary = pd.DataFrame(
23     rows,
24     columns=["Model", "Accuracy", "Precision", "Recall", "F1"]
25 )
26
27 print(summary)

```

This creates a summary table showing all the metrics for both the models with and without PCA.

	Model	Accuracy	Precision	Recall	F1
0	Logistic (NO PCA)	0.675978	0.888889	0.222222	0.355556
1	SVM (NO PCA)	0.201117	0.705882	1.000000	0.827586
2	Logistic (WITH PCA)	0.692737	0.717949	0.388889	0.504505
3	SVM (WITH PCA)	0.687151	0.710526	0.375000	0.490909

- **Logistic Regression**

- Without PCA, the logistic regression model was too cautious. It rarely predicted class 1. This is indicated by the high precision but very low recall.
- PCA gives the model a cleaner low-dimensional structure.
- It predicts 1 more often, improving recall while slightly reducing precision.
- PCA improves the model's balance and effectiveness.

- **SVM**

- Before PCA, SVM collapsed and predicted every sample as class 1. This can be seen as the recall is 1. Hence, the accuracy was very poor (0.20).
- This is a failed model.
- After PCA, the dimensionality reduces and the optimization stabilizes.
- The SVM model learns an actual linear separating hyperplane.
- The predictions become balanced and accuracy improves massively.

Logistic Regression becomes more balanced with PCA, while SVM becomes drastically more stable and accurate.